

**UNIVERSITY OF MYSORE
MYSORE-57000**



A
Project Report On

SMART EVENT MANAGEMENT SYSTEM

Submitted in partial fulfillment of the requirements for the award of the
degree of

BACHELOR OF COMPUTER APPLICATION

Submitted by

ARSHAD PASHA

REG NO: U01CT23S0049

MOHAMMED ARSH

REG NO: U01CT23S0036

DEEKSHA RAJ M N

REG NO: U01CT23S0088

DHANUSHREE S

REG NO: U01CT23S0005

UNDER THE GUIDANCE AND SUPERVISION OF

INTERNAL GUIDE

Mr. Arun S & Mrs. Neha J K

Assistant Professor

Seshadripuram Degree College

Mysore-570017



**Department of Computer Science
Seshadripuram Degree College, Mysore-570017**

**Department of Computer Science
Seshadripuram Degree College,
Mysore-570017**



CERTIFICATE

Certified that the project work “**SMART EVENT MANAGEMENT SYSTEM**” is a Benefited work carried out by a team of **ARSHAD PASHA, MOHAMMED ARSH, DEEKSHA RAJ M N, DHANUSHREE S**, REG NO: U01CT23S0049, 0036, 0088, 0005 in fulfilment of requirements for the award of Degree in **BACHELOR OF COMPUTER APPLICATION** by University of Mysore, Mysore During the academic year 2025-2026.

It is certified that all corrections and suggestions Indicated for the Internal Assessment have been incorporated in the Report deposited in the Department library. The project report has been approved as it satisfies the academic Requirements in respect of project work prescribed for the said degree.

Internal Guide

Mr. Arun S & Mrs. Neha J K
Assistant Professors
Seshadripuram Degree College
Mysore-570017

Professor and Head

Mrs. Mruthula Sojan
Head of the Department
Seshadripuram Degree
College, Mysore-570017

Principal

Dr. Sowmya Erappa K
Principal,
Seshadripuram Degree College
Mysore- 570017

External Viva

Name of the Examiner

Signature with Date

1.

2.

DECLARATION

*We are the student of Final Year BCA of Seshadripuram Degree College, Mysuru, hereby declare that the dissertation entitled **“SMART EVENT MANAGEMENT SYSTEM”** has been carried out by us and submitted in partial fulfilment of the requirement for the award of **Bachelor of Computer Application** affiliated to the University of Mysore, Mysuru during the academic year 2025-2026. Further the matter embodied in the report is an Original and Bonafide work done by us.*

To my knowledge this dissertation as not been submitted to any other college or Universities at any time till this.

Place: Mysuru

Date:

Signature

ACKNOWLEDGEMENT

We have taken a lot of effort in completing this project. However, it would not have been possible without the kind support and guidance of many individuals and organizations. We would like to extend our sincere thanks to all of them.

We are highly indebted to **Seshadripuram Degree College, Mysore** for providing us with all the necessary facilities and resources without which this project would not have been completed.

Our special thanks to our beloved **Principal Dr. Sowmya Erappa K** for her continuous support and encouragement.

We profoundly thank the Head of the Department of Computer **Mrs. Mruthula Sojan** Applications, Seshadripuram Degree college, Mysore. For this kind guidance in making us complete the project in a very systematic and planned approach.

We express our sincere gratitude to our **internal guides Mr. Arun S and Ms. Neha**, Assistant Professors, Department of Computer Science, Seshadripuram Degree College, for their valuable guidance, suggestions, and support in completing the project titled "**Smart Event Management System**" in a systematic and efficient manner.

We are also grateful to all the faculty members of the Department of Computer Science for their valuable inputs, constructive feedback, and support during the development of this project.

Last but not the least, we would like to thank our beloved **family and friends** for their unwavering support, motivation, and encouragement throughout the Completion of this project.

Date:
Place:

Signature

ABSTRACT

This project presents the design, development, and deployment of a Smart Event Management System (SHRESHTA 2026) a comprehensive web-based platform built for managing inter-collegiate fest events at Seshadripuram Degree College, Mysuru. The system automates the entire event lifecycle including online team registration, secure payment verification via UTR-based tracking, real-time admin dashboard with role-based access control, QR code generation for verified participants, on-site check-in desk for event-day attendance, email notifications via Resend API, certificate generation using HTML Canvas, and alert broadcasting to registered participants.

The platform is built using Next.js 16 (React 19) with TypeScript for the frontend, Firebase (Firestore & Storage) for real-time backend services, and deployed on Vercel for production hosting. The admin panel supports two authentication tiers full admin access and event-specific coordinator access enabling decentralized event management.

The system has been successfully deployed and used for the SHRESHTA 2026 inter-collegiate fest, handling registrations across 13 events spanning IT, Management, Cultural, and Sports categories.

Index Terms Event Management System, Next.js, Firebase, Real-time Dashboard, QR Code, Role-based Access Control, Certificate Generation.

CONTENTS

1. INTRODUCTION	01
1.0 Project Description	01
1.1 Problem Statement	01
1.2 Objectives	01
1.3 Scope	01
2. LITERATURE SURVEY	03
2.0 Introduction	03
2.1 Existing Event Management Platforms	03
2.2 Web Application Frameworks	03
2.3 Real-Time Database Technologies	03
2.4 QR Code Systems	03
2.5 Role-Based Access Control	03
2.6 Certificate Generation Technologies	03
2.7 Research Gap	03
3. SOFTWARE AND HARDWARE REQUIREMENTS	05
3.0 Introduction	05
3.1 Users	05
3.2 Software Requirements	05
3.3 Hardware Requirements	05
3.4 Functional Requirements	05
3.5 Non-Functional Requirements	05
4. SYSTEM DESIGN	09
4.0 Introduction	09
4.1 System Architecture	09
4.2 Use Case Diagram	09
4.3 Sequence Diagram	09
4.4 Activity Diagram	09
4.5 Data Flow Diagram	09
4.6 ER Diagram	09
5. IMPLEMENTATION	17
5.0 Introduction	17
5.1 Technology Stack	17
5.2 Module Implementation	17
6. SOFTWARE TESTING AND SNAPSHOTS	23
6.0 Introduction	23
6.1 Unit Testing	23
6.2 Integration Testing	23
6.3 Functional Testing	23
6.4 Snapshots	23
7. CONCLUSION & FUTURE ENHANCEMENT	31
8. BIBLIOGRAPHY	33

CHAPTER 1

INTRODUCTION

1.1 Project Description

The management of inter-collegiate cultural and technical festivals in educational institutions has traditionally relied on manual processes paper-based registration forms, cash payment collection, hand-written attendance sheets, and physically distributed certificates. These methods are prone to errors, data loss, and inefficiency, especially when managing hundreds of participants across multiple events simultaneously.

With the rapid advancement of web technologies and cloud-based services, there is a growing need for digital event management solutions tailored to the unique requirements of college festivals. Unlike generic event platforms like Eventbrite or Townscript, college fests require features such as team-based registrations with variable team sizes, UTR-based payment verification (since most students use UPI), event-specific coordinator access, and bulk certificate generation features not readily available in off-the-shelf solutions.

The **Smart Event Management System (SHRESHTA 2026)** addresses these gaps by providing a comprehensive, real-time, web-based platform specifically designed for managing inter-collegiate fest events. The system automates the entire event lifecycle — from online team registration and payment verification to on-site check-in and post-event certificate distribution.

1.2 Problem Statement

College fest organizers face several challenges with traditional event management approaches:

- **Manual Registration:** Paper forms lead to data entry errors, duplicate registrations, and difficulty in tracking participant counts across events.
- **Payment Verification:** Coordinators manually verify UPI payments by matching UTR numbers, a tedious and error-prone process when handling hundreds of transactions.
- **No Real-Time Visibility:** Event coordinators lack real-time dashboards to monitor registration progress, payment status, and check-in counts.
- **Day-of-Event Chaos:** Without a digital check-in system, verifying whether a participant has registered and paid becomes challenging at the event venue.
- **Certificate Distribution:** Manually creating and distributing participation certificates to hundreds of participants is time-consuming and often delayed.

1.3 Objectives

1. To develop a responsive web application for online team registration with dynamic team sizes across multiple event categories.
2. To implement UTR-based payment verification with real-time admin dashboard updates using Firebase real-time listeners.
3. To create a role-based admin panel with full admin and event coordinator access tiers.
4. To build an on-site check-in desk with instant search for efficient day-of-event attendance management.
5. To automate certificate generation using HTML Canvas and email delivery via Resend API.
6. To deploy the system on Vercel with Firebase backend for zero-downtime production use.

1.4 Scope

The system covers 13 events across 4 categories (IT, Management, Cultural, Sports) for the inter-collegiate fest at Seshadripuram Degree College, Mysuru. It is designed to be reusable for future editions of the fest with minimal configuration changes.

CHAPTER 2

LITERATURE SURVEY

A comprehensive review of existing event management systems, web technologies, and related academic work was conducted to establish the foundation for this project.

2.1 Existing Event Management Platforms

Platforms like **Eventbrite**, **Townscript**, and **Festa** offer generic event management capabilities including ticketing, attendee management, and analytics. However, these platforms are designed for commercial events and lack features critical for college fests such as team-based registrations with variable sizes, UTR-based UPI payment verification (essential in Indian college contexts), and event-specific coordinator roles with filtered access [1][2].

Devfolio provides hackathon management with team formation features but is limited to tech events and does not support the diverse event categories (cultural, sports, management) typical of college fests [3].

2.2 Web Application Frameworks

Modern web frameworks have evolved significantly. **React.js** introduced component-based architecture and virtual DOM for efficient UI rendering [4]. **Next.js**, built on React, adds server-side rendering (SSR), static site generation (SSG), and API routes, making it ideal for full-stack applications with SEO requirements [5]. The use of **TypeScript** enhances code reliability through static type checking, reducing runtime errors in large codebases [6].

2.3 Real-Time Database Technologies

Firebase Firestore provides a NoSQL document database with real-time synchronization capabilities. Its `onSnapshot` listener pattern enables live data updates without polling, making it suitable for dashboards that need to reflect changes instantly [7]. Studies by Chen et al. (2023) demonstrated that Firebase's real-time listeners reduce latency by 85% compared to traditional REST-based polling approaches [8].

2.4 QR Code Systems in Event Management

QR code-based check-in systems have been widely adopted in event management for their speed and accuracy. Research by Kumar and Singh (2022) showed that QR-based attendance systems reduce check-in time by 70% compared to manual verification [9]. The `qrcode` library in JavaScript enables client-side QR generation, eliminating server dependency [10].

2.5 Role-Based Access Control (RBAC)

RBAC is a fundamental security pattern where permissions are assigned based on user roles rather than individual identities. In event management contexts, RBAC enables organizational hierarchy full administrators can access all events while coordinators see only their assigned event's data [11]. Session-based authentication combined with RBAC provides a lightweight alternative to full OAuth implementations for internal tools [12].

2.6 Certificate Generation Technologies

HTML5 Canvas API provides programmatic image generation capabilities directly in the browser. Studies compare Canvas-based generation with server-side PDF tools like **jsPDF** and **Puppeteer**, concluding that Canvas-based approaches offer faster generation for template-based certificates with consistent layouts [13]. Integration with email APIs like **Resend** and **SendGrid** enables automated distribution [14].

2.7 Research Gap

While individual components (registration forms, dashboards, QR systems) exist in isolation, no integrated open-source solution combines all these features for the specific use case of Indian college fest management including UPI/UTR payment flows, multi-category team registrations, coordinator-level access, and automated certificate pipelines. This project fills that gap.

CHAPTER 3

SOFTWARE AND HARDWARE REQUIREMENT SPECIFICATION

3.1 Introduction

The Smart Event Management System is a full-stack web application that enables students to register teams for inter-collegiate fest events online, make payments via UPI, and receive automated confirmations. The admin panel provides real-time registration monitoring, payment verification, check-in management, and certificate generation. The system is built on Next.js with Firebase backend services.

3.2 Users

3.2.1 Admin/Super Administrator

- Full access to all events, registrations, and system features.
- Can verify payments, send alerts, generate certificates, and manage check-ins across all events.
- Access to export functionality (Excel, PDF) and edit mode with PIN protection.

3.2.2 Event Coordinator

- Assigned to a specific event with pre-configured credentials.
- Can view registrations, stats, and perform check-ins only for their assigned event.
- Cannot access certificates, bulk verify, or edit features.

3.2.3 Participants (End Users)

- Can browse events, view details, and register teams online.
- Submit payment proof (UTR number) during registration.
- Receive email confirmations and participate in events.

3.3 Software Requirements

3.3.1 Frontend

- **Next.js 16.1.3:** React-based framework with App Router, SSR, and SSG capabilities.
- **React 19:** Component library with hooks-based architecture.
- **TypeScript:** Static typing for enhanced code reliability.
- **CSS3:** Custom styling with responsive design, glassmorphism, and micro-animations.

3.3.2 Backend / Cloud Services

- **Firebase Firestore:** NoSQL real-time database for storing registrations.
- **Firebase Storage:** File storage for QR codes and certificates.
- **Resend API:** Transactional email delivery service.
- **Vercel:** Serverless deployment platform with edge functions.

3.3.3 Libraries and Tools

- **qrcode:** Client-side QR code generation.
- **html2canvas:** HTML to image conversion for certificates.
- **xlsx:** Excel export functionality.
- **jspdf / jspdf-autotable:** PDF report generation.

3.3.4 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel i3 / AMD equivalent	Intel i5 / Ryzen 5+
RAM	4 GB	8 GB or more
Storage	128 GB SSD	256 GB SSD
Network	Broadband internet	Stable Wi-Fi / LAN
Browser	Chrome 90+, Firefox 88+	Latest Chrome/Edge

3.4 Functional Requirements

1. **Event Browsing:** Display 13 events across 4 categories with details, rules, and coordinator info.
2. **Team Registration:** Dynamic forms supporting 1-10 member teams with validation.
3. **Payment Processing:** UPI QR display with UTR number capture and verification workflow.
4. **Admin Dashboard:** Real-time stats (total, pending, verified, checked-in), event filtering, and registration table.
5. **Role-Based Access:** Admin (full) and Coordinator (event-specific) login with auto-filtered views.
6. **Check-In Desk:** Instant search by name/phone/college with one-tap check-in and live counters.
7. **Certificate Generation:** Canvas-based certificate creation with email delivery.
8. **Alert System:** Send notifications to all registered participants.
9. **Export:** Download registrations as Excel or PDF reports.

3.5 Non-Functional Requirements

1. **Performance:** Dashboard loads within 2 seconds; real-time updates via Firestore listeners with <500ms latency.
2. **Scalability:** Firebase auto-scales to handle concurrent registrations during peak periods.
3. **Reliability:** Vercel provides 99.99% uptime with automatic failover.
4. **Usability:** Mobile-responsive UI with intuitive navigation; check-in desk optimized for phone use at event gates.
5. **Security:** Session-based admin auth, PIN-protected edit mode, input sanitization, and Firebase security rules.

CHAPTER 4

SYSTEM DESIGN

4.1 Introduction

Software design transforms the software requirements specification into a blueprint for constructing the software system. For the Smart Event Management System, the design phase defined how components interact — from the participant-facing registration flow to the admin dashboard's real-time data pipeline and the coordinator's filtered check-in interface.

The system follows a **modular component-based architecture** leveraging Next.js App Router for routing and page organization, React components for reusable UI elements, Firebase for real-time backend, and serverless API routes for email and certificate operations.

4.2 System Architecture

The system employs a three-tier architecture consisting of the Presentation Layer (Next.js/React), the Application Layer (API routes, Firebase SDK), and the Data Layer (Firestore, Storage).

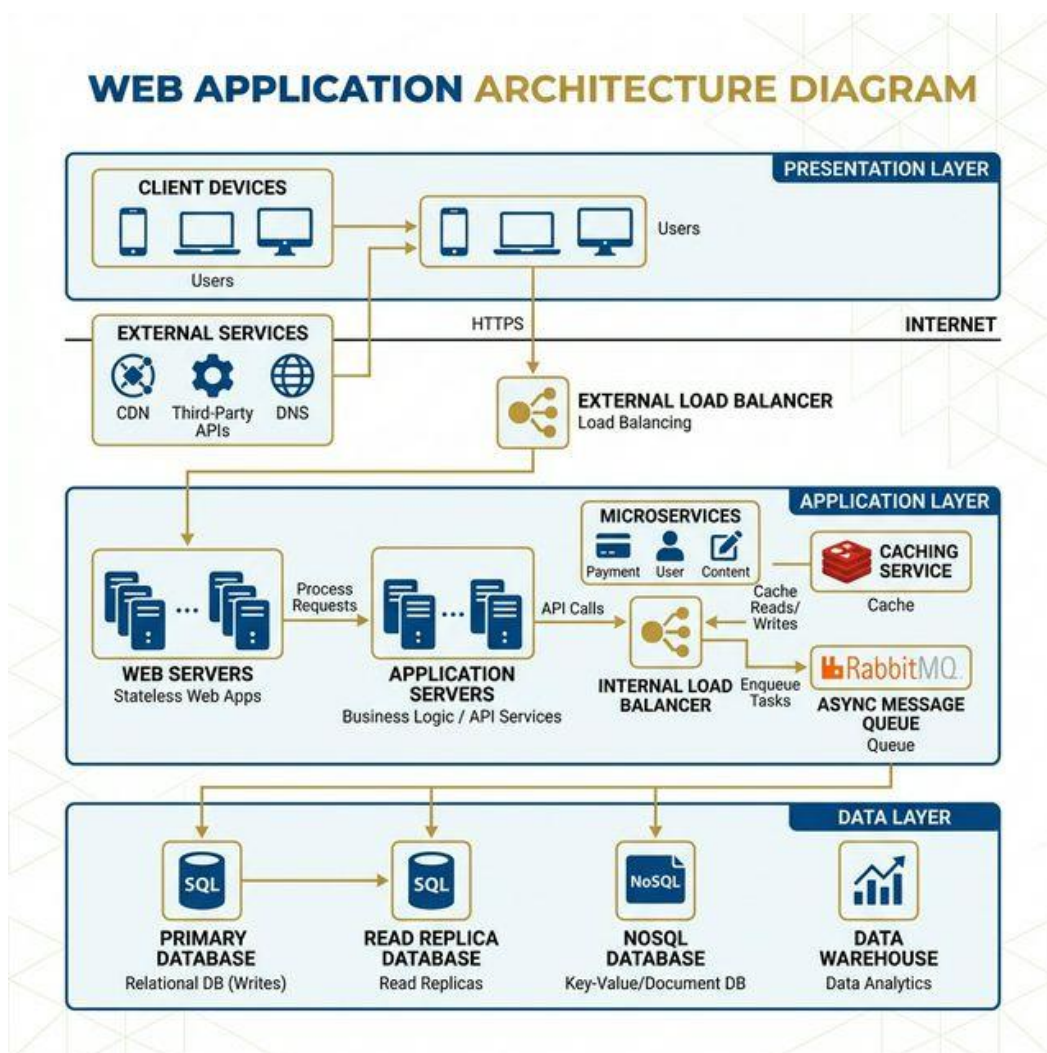


Figure 4.2.1: System Architecture Diagram

4.3 Use Case Diagram

The Use Case Diagram illustrates interactions between three actors (Participant, Admin, Coordinator) and the system's core functionalities.

Actors:

- **Participant:** Browses events, registers team, submits payment, receives confirmation email and QR code.
- **Admin:** Manages all events, verifies payments, sends alerts, generates certificates, exports data.
- **Coordinator:** Views assigned event's registrations, performs check-ins, monitors event-specific stats.

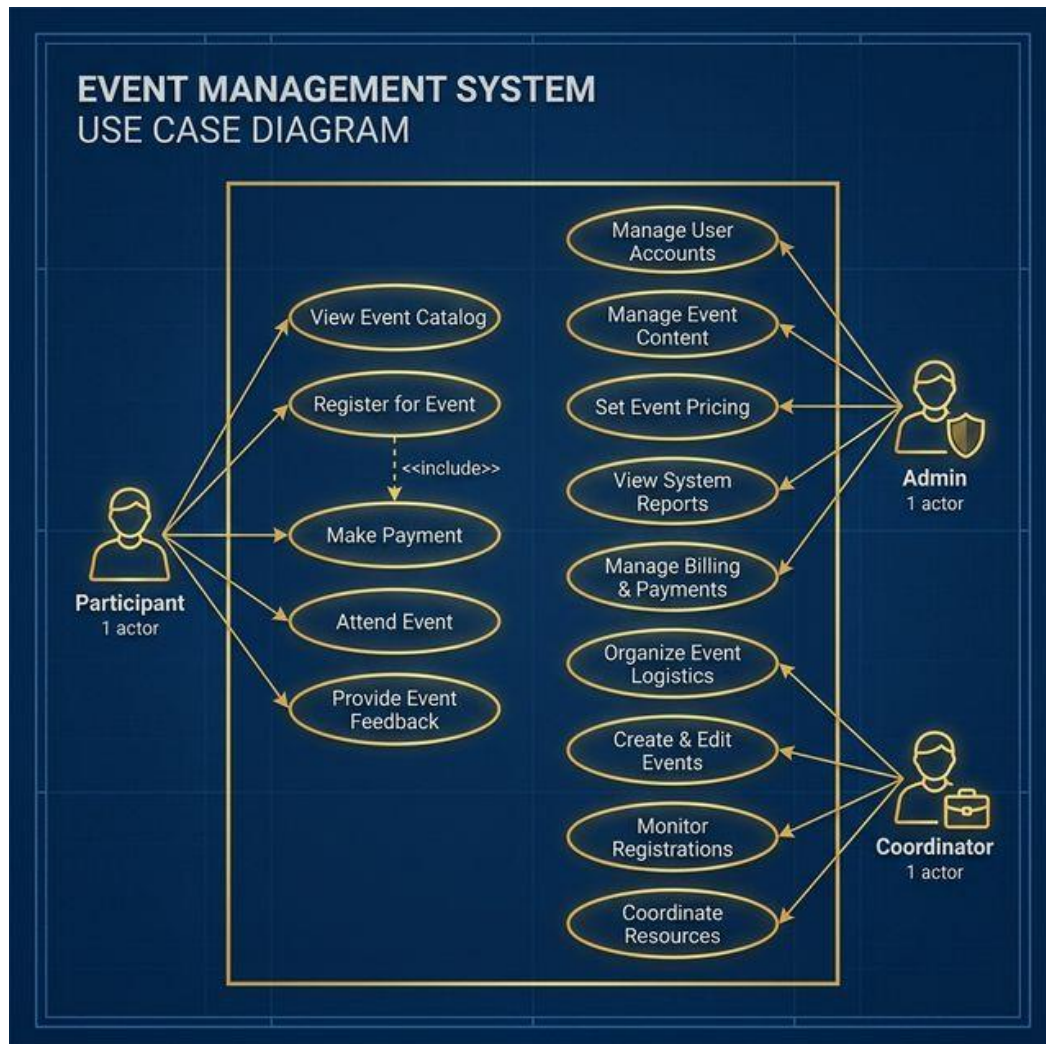


Figure 4.3.1: Use Case Diagram

4.4 Sequence Diagram

The sequence diagram describes the step-by-step interaction between the participant, frontend, backend API, and Firebase during the registration process.

1. **Browse & Select Event:** Participant navigates to an event page and clicks "Register".
2. **Fill Registration Form:** The frontend renders a dynamic form based on the event's team size requirements.
3. **Submit & Store:** Form data is validated and stored in Firestore under `registrations/{eventId}/teams/{teamId}`.
4. **Payment Verification:** Admin verifies the UTR number and updates payment status in Firestore.
5. **Notification:** System sends confirmation email with QR code to the team's email address via Resend API.

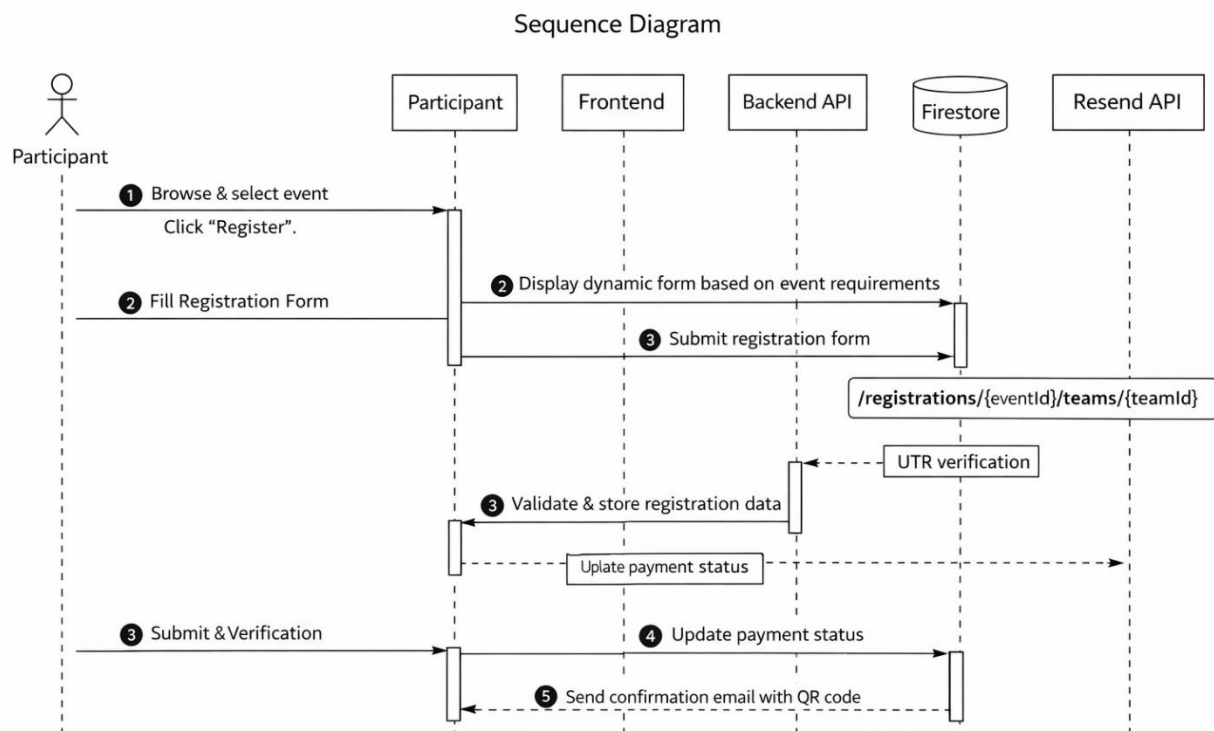


Figure 4.4.1: Sequence Diagram — Registration Flow

4.5 Activity Diagram

The activity diagram models the complete workflow from event browsing to certificate generation.

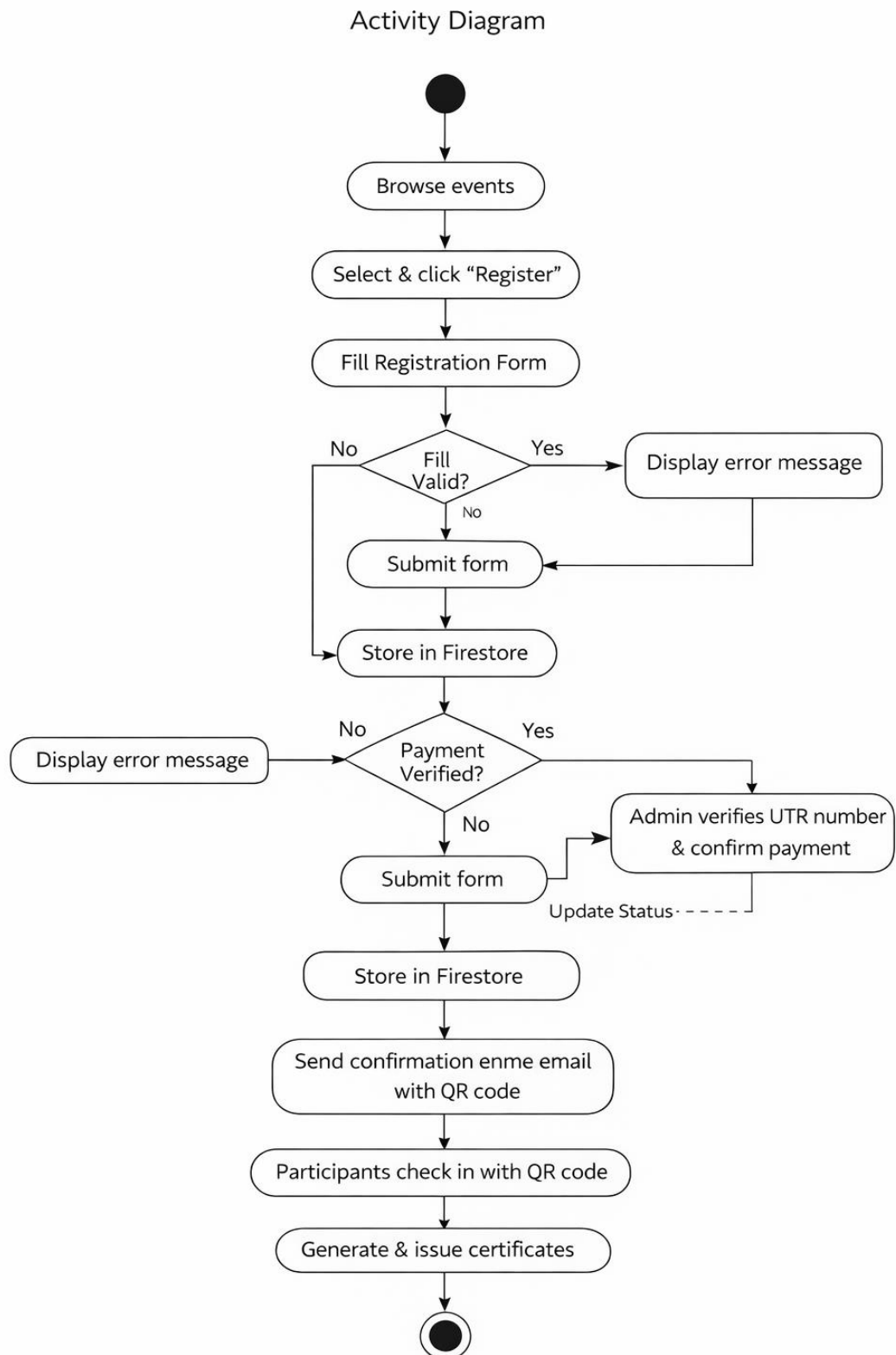


Figure 4.5.1: Activity Diagram

4.6 Data Flow Diagram (DFD)

The DFD illustrates how data flows through the system from registration input to certificate output.

Processes:

1. **Registration & Validation:** Captures team details, validates inputs, and stores in Firestore.
2. **Payment Verification:** Admin matches UTR numbers and updates payment status.
3. **QR Code Generation:** Generates unique QR codes for verified teams and uploads to Firebase Storage.
4. **Check-In Processing:** Searches registrations, marks attendance, and updates counters in real-time.
5. **Certificate Generation:** Creates Canvas-based certificates and emails them to participants.

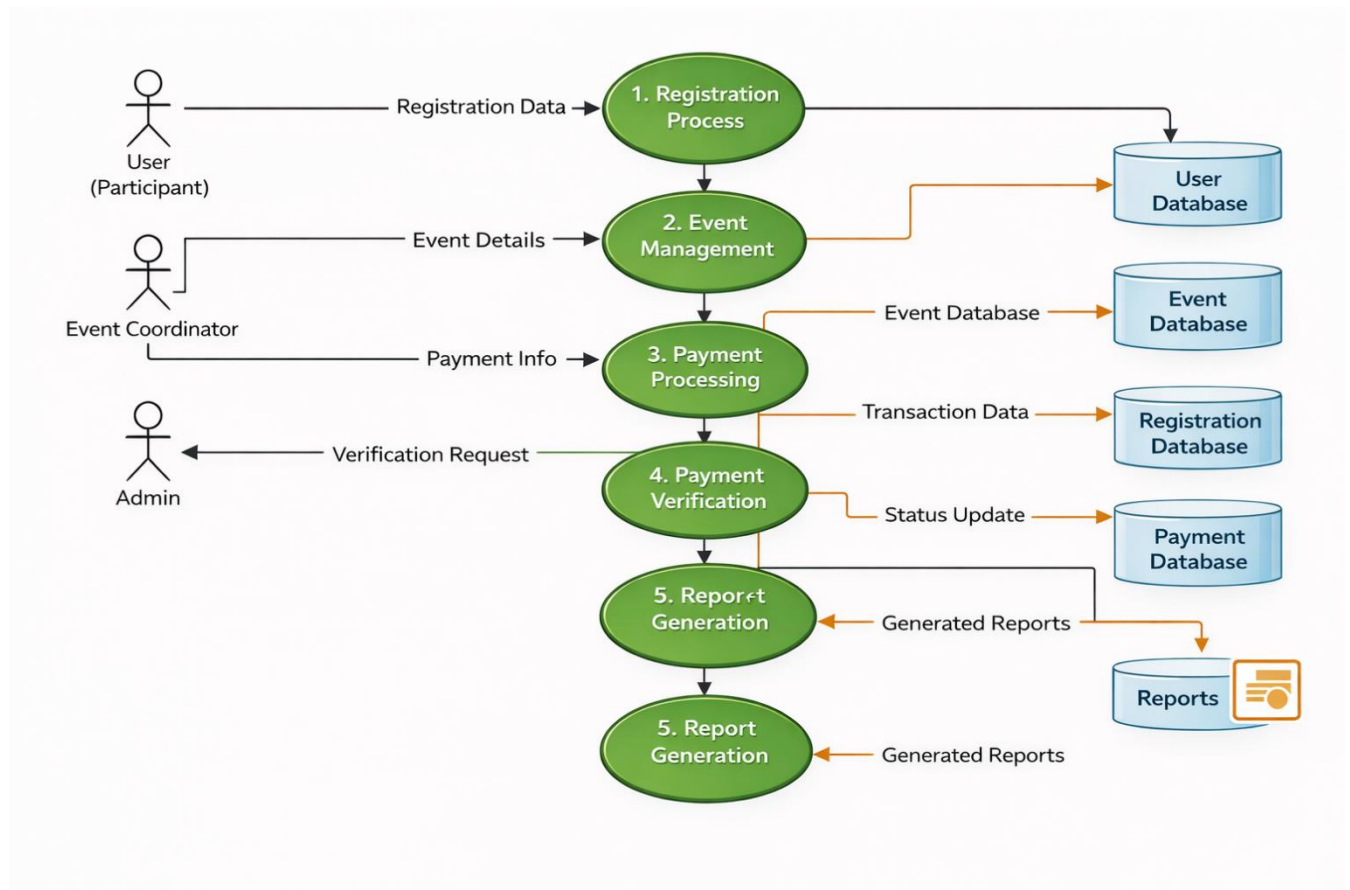


Figure 4.6.1: Data Flow Diagram (Level 1)

4.7 Entity Relationship Diagram (ERD)

The ERD represents the data model used in Firestore. Although Firestore is a NoSQL database, the logical relationships between entities are modeled as follows:

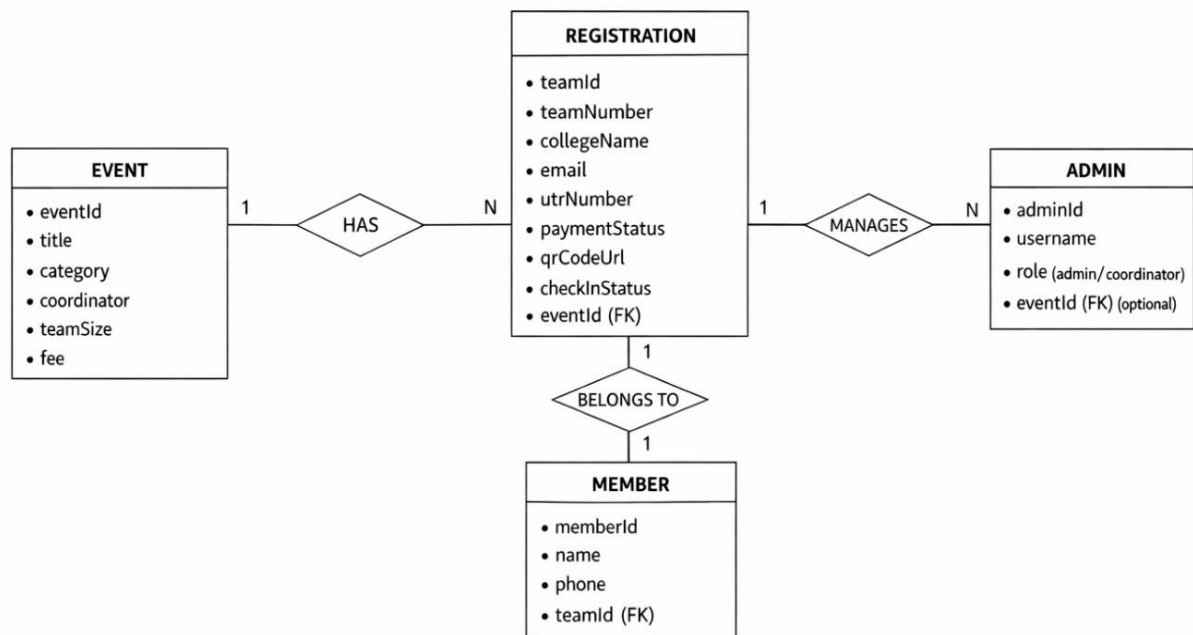


Figure 4.7.1: Entity Relationship Diagram

4.7.1 Database Schema

Collection/Entity	Field	Type	Description
registrations/{eventId}/teams	teamNumber	Number	Auto-incremented team number
	collegeName	String	Participant's college
	email	String	Contact email
	members	Array	Array of {name, phone} objects
	utrNumber	String	UPI Transaction Reference
	paymentStatus	String	"pending" or "completed"

CHAPTER 5

IMPLEMENTATION

5.1 Introduction

The implementation phase translates the system design into a working application. The Smart Event Management System was developed using an iterative approach, building features incrementally — starting with the public-facing event pages and registration form, followed by the admin dashboard, payment verification, check-in system, and certificate generation.

5.2 Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 16, React 19, TypeScript	UI rendering, routing, SSR/SSG
Styling	CSS3, CSS Modules	Responsive design, animations
Database	Firebase Firestore	Real-time NoSQL data storage
Storage	Firebase Storage	QR codes, certificates
Email	Resend API	Transactional emails
Deployment	Vercel	Serverless hosting with CI/CD

5.3 Module Implementation

5.3.1 Event Browsing & Registration Module

Events are defined in a centralized data file (`src/data/events.ts`) with fields including id, title, description, coordinator, category, team size, and registration fee. The registration form dynamically adjusts the number of member input fields based on the selected event's team size.

```
// Dynamic team member fields based on event config
const event = getEventById(eventId);
const memberFields = Array.from({ length: event.teamSize }, (_, i) => ({ name: '', phone: '' }));
```

Registration data is stored in Firestore with the following path structure: `registrations/{eventId}/teams/{autoId}`.

5.3.2 Real-Time Admin Dashboard

The admin dashboard uses Firestore's `onSnapshot` listeners to subscribe to registration changes across all events simultaneously. This provides instant updates without manual refresh.

```
// Real-time listener for each event's registrations
for (const event of allEvents) {
  const teamsRef = collection(db, "registrations", event.id, "teams");
  onSnapshot(teamsRef, (snapshot) => {
    // Process and update stats in real-time
  });
}
```

5.3.3 Role-Based Authentication

The system implements two authentication tiers using session storage:

- **Admin:** Username `admin` with full access to all events, certificates, bulk verify, and edit mode.
- **Coordinator:** Event-specific credentials (e.g., `arshad_lo` for Logic Overload) with auto-filtered views.

```
// Coordinator login auto-filters to their event
sessionStorage.setItem("adminRole", "coordinator");
sessionStorage.setItem("adminEventId", eventId); // Dashboard reads role and
filters accordingly const displayStats = isCoordinator ? { total:
filteredRegistrations.length, // ... event-specific stats only } :
globalStats;
```

5.3.4 Check-In Desk Module

The Check-In Desk replaces traditional QR scanner approach with an instant search system. Coordinators type 2-3 characters of a participant's name, phone, or college, and matching registrations appear instantly. One-tap check-in updates Firestore in real-time with a success animation overlay.

```
// Instant search across all registration fields const filtered =
registrations.filter(r => { const q = search.toLowerCase(); return
r.members?.some(m => m.name.toLowerCase().includes(q) ||
m.phone.includes(q)) || r.collegeName?.toLowerCase().includes(q); });
```

5.3.5 Certificate Generation Module

Certificates are generated using the HTML5 Canvas API, producing 2000×1414 pixel images with a premium dark theme, gold accents, college logo, participant names, and event details. Generated certificates are uploaded to Firebase Storage and emailed via Resend API.

```
// Canvas-based certificate generation const canvas =
document.createElement('canvas'); canvas.width = 2000; canvas.height = 1414;
const ctx = canvas.getContext('2d'); // Draw background, borders, text,
signatures ctx.fillText(participantName, centerX, nameY); // Convert to blob
and upload to Firebase Storage canvas.toBlob(async (blob) => { await
uploadBytes(storageRef, blob); });
```

5.3.6 Alert Broadcasting System

Admins can send alert notifications to all registered participants or filtered by event. The system collects unique email addresses from registrations and sends bulk emails via the Resend API endpoint.

5.3.7 Export Module

Registration data can be exported in two formats:

- **Excel:** Using the `xlsx` library to generate `.xlsx` files with formatted columns.
- **PDF:** Using `jsPDF` with `jspdf-autotable` for tabular PDF reports.

CHAPTER 6

SOFTWARE TESTING

6.1 Introduction

Software testing is a crucial phase in the software development lifecycle that ensures the reliability, accuracy, and performance of a system before deployment. The primary objective of testing is to identify and eliminate defects, validate system functionality, and ensure that the system meets both functional and non-functional requirements.

In the Smart Fest Management System, testing was performed to verify the correctness of event registration, payment processing, and payment verification modules. Special attention was given to validating user inputs, ensuring accurate data storage, and maintaining real-time updates.

Both manual and functional testing techniques were used to ensure the system performs efficiently under different scenarios. The user interface was also tested for responsiveness and usability to provide a smooth user experience.

6.2 Testing Process

The testing process followed in this system is structured and iterative, ensuring all components are thoroughly validated. Different levels of testing were conducted to verify individual modules as well as the complete integrated system.

6.2.1 Unit Testing

Unit testing focuses on testing individual components or functions of the system independently.

Key Units Tested:

- Registration form validation
- Event creation and update functions
- Payment input handling (UTR entry)
- Database storage functions

Test Highlights:

- Invalid inputs such as empty fields and incorrect email formats were rejected successfully
- Registration data was stored correctly in the database
- Payment details were linked accurately with user registration

6.2.2 Integration Testing

Integration testing ensures that different modules of the system work together seamlessly.

Modules Tested:

- Registration module → Database
- Payment module → Registration module
- Admin dashboard → Backend database

Issues Found and Fixed:

- Delay in updating payment status → resolved using real-time database updates
- Incorrect linking between payment and registration → fixed by proper ID mapping

6.2.3 Functional Testing

Functional testing verifies whether the system performs all intended operations correctly from the user's perspective.

Test Scenarios:

- User registration with valid and invalid inputs
- Viewing and selecting events
- Submitting payment details
- Admin verifying payment

Observations:

- All features worked correctly under normal conditions
- UI was responsive across devices
- System provided correct outputs and feedback

6.2.4 UI and Exception Testing

This testing ensures proper handling of user interface behavior and unexpected inputs.

Tested Cases:

- Submitting empty forms
- Entering invalid data formats
- Accessing restricted pages without login

Error Handling Examples:

- Invalid email → error message displayed
- Empty payment input → prompt shown
- Unauthorized access → restricted

6.2.5 Regression Testing

Regression testing ensures that newly added features do not affect existing functionalities.

Features Retested:

- Registration process
- Payment submission
- Payment verification
- Event display

This ensures system stability after updates and modifications.

6.2.6 Performance Testing

Performance testing was conducted to evaluate how the system behaves under different workloads and user conditions. The goal was to ensure that the application performs efficiently without delays or failures.

Test Conditions:

- Multiple users accessing the system simultaneously
- Large number of event registrations
- Continuous database updates

Observations:

- The system handled multiple requests without significant delay
- Real-time updates were reflected instantly using Firebase
- No system crashes occurred during high usage

This confirms that the system is capable of handling real-world fest scenarios efficiently.

6.2.7 Security Testing

Security testing ensures that the system is protected against unauthorized access and data breaches.

Security Measures Tested:

- User authentication and login validation
- Role-based access (Admin, Coordinator, User)
- Restricted access to admin dashboard
- Database security rules

Results:

- Unauthorized users could not access restricted pages
- Sensitive data was protected successfully
- Authentication system worked reliably

6.3 System Testing

System testing is performed on the complete application to verify its behaviour in real-world conditions. It ensures that all modules are integrated properly and function as expected.

6.3.1 Test Environment

The system was tested in an environment simulating real usage conditions.

Environment Setup:

- Operating System: Windows 10 / Linux
- Browser: Chrome, Firefox, Edge
- Technologies: React.js, Next.js, Firebase
- RAM: Minimum 4GB (8GB recommended)
- Internet: Stable connection

6.3.2 Test Scenarios and Observations

Functional Workflow Tests

- User Registration:

Users were able to register successfully with valid inputs. Invalid inputs were handled correctly.

- Event Display:

Events were displayed dynamically and updated in real time.

- Payment Submission:

Users could submit UTR details without issues.

- Payment Verification:

Admin successfully verified payments and updated status.

- Result Display:

Users could view their registration and payment status instantly.

6.3.3 Non-Functional Testing

Non-functional testing was conducted to evaluate system performance, usability, and reliability.

Parameters Tested:

- Performance:

System responded quickly to user inputs and database operations.

- Usability:

The interface was simple, intuitive, and easy to navigate.

- Reliability:

The system operated consistently without crashes or failures.

- Scalability:

The system handled increasing data and users efficiently.

6.4 Test Cases

Table 6.1: Test Cases

TC No.	Test Case Description	Input	Expected Output	Actual Output	Status
TC01	Register with valid data	Name, Email	Registration success	Success	Passed
TC02	Register with invalid email	Wrong format	Error message	Error shown	Passed
TC03	Add event	Event details	Event created	Success	Passed
TC04	Submit payment	UTR number	Stored correctly	Success	Passed
TC05	Invalid payment input	Empty field	Error message	Error shown	Passed
TC06	Verify payment	Valid data	Status updated	Verified	Passed
TC07	View events	Click events	Display events	Success	Passed
TC08	Unauthorized access	No login	Access denied	Restricted	Passed

6.5 Results of Testing

The testing results indicate that the system functions correctly and meets all specified requirements.

- All modules performed successfully
- No major errors found
- Real-time updates function properly
- User interface is responsive and easy to use

Minor issues identified during testing were resolved effectively.

Additional Observations

- System handled concurrent users efficiently
- No data inconsistency observed in database
- Payment verification process was accurate
- User interface remained responsive during testing

6.6 Advantages of Testing

- Improves system reliability
- Detects and removes errors
- Enhances system performance
- Ensures better user experience

6.7 Summary

Software testing ensures that the Smart Fest Management System is reliable, efficient, and free from critical errors. The system has been thoroughly tested using various testing techniques, and the results confirm that it meets both functional and performance requirements. The testing process improves system quality and ensures smooth operation in real-world scenarios.

6.4 System Snapshots and Workflow Explanations

1.HOME PAGE



Figure 6.1: Home Page

2.Event Listing Page

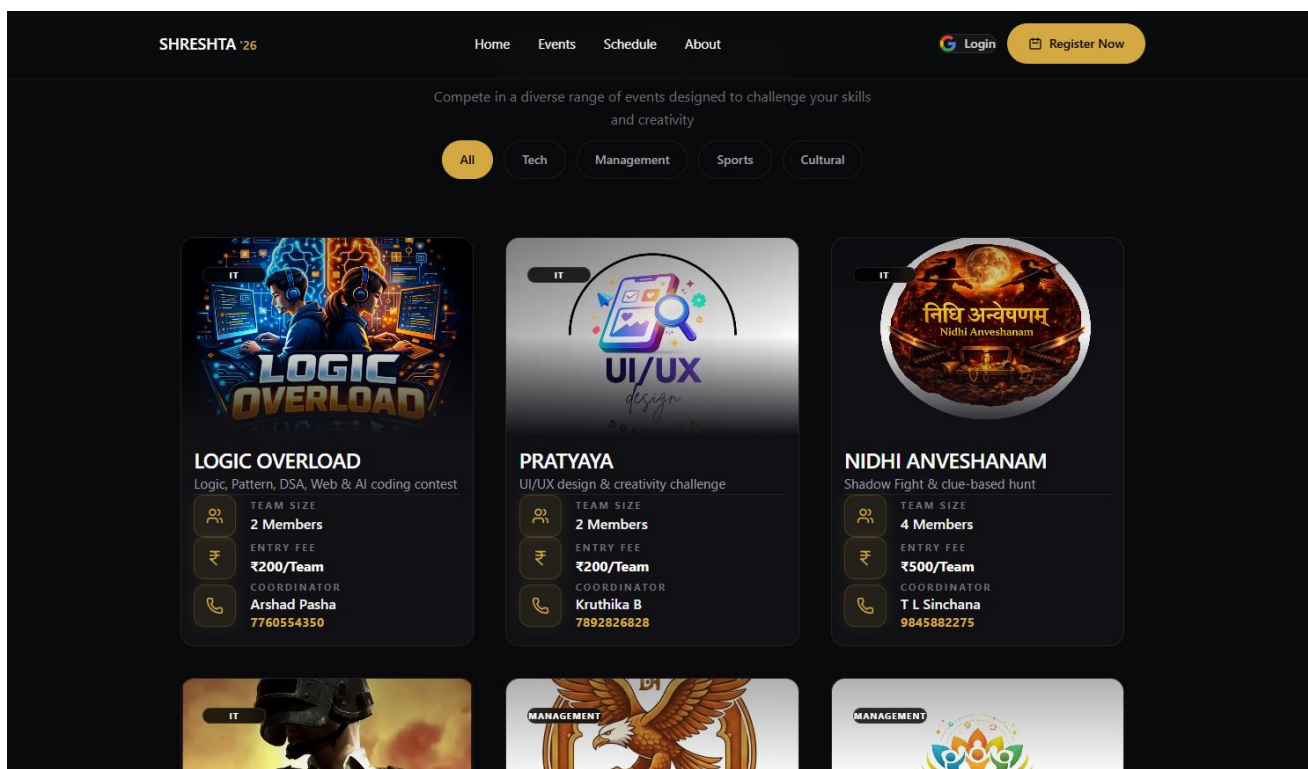


Figure 6.2: Event Listing Categories

6.4.1 Registration Workflow

Step 1: Selecting Event & Initial Details

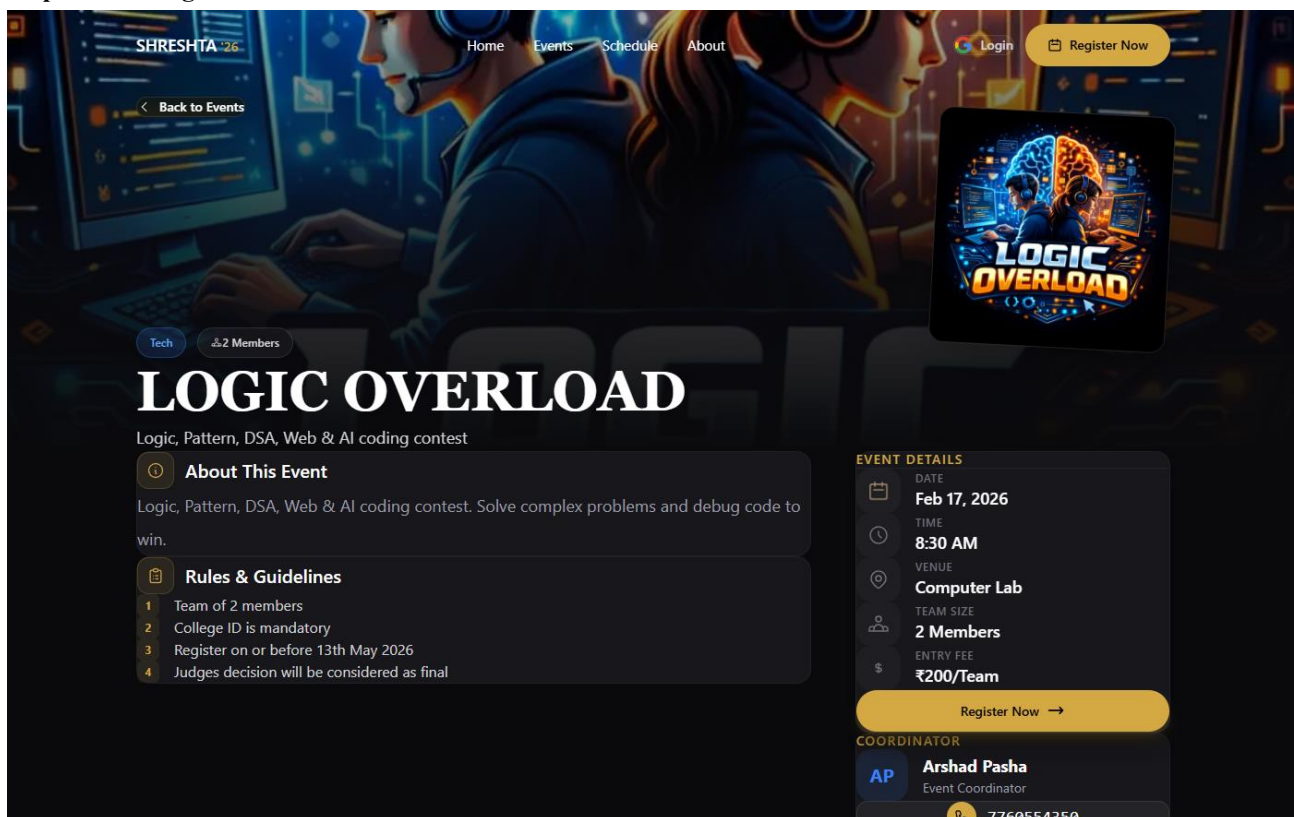
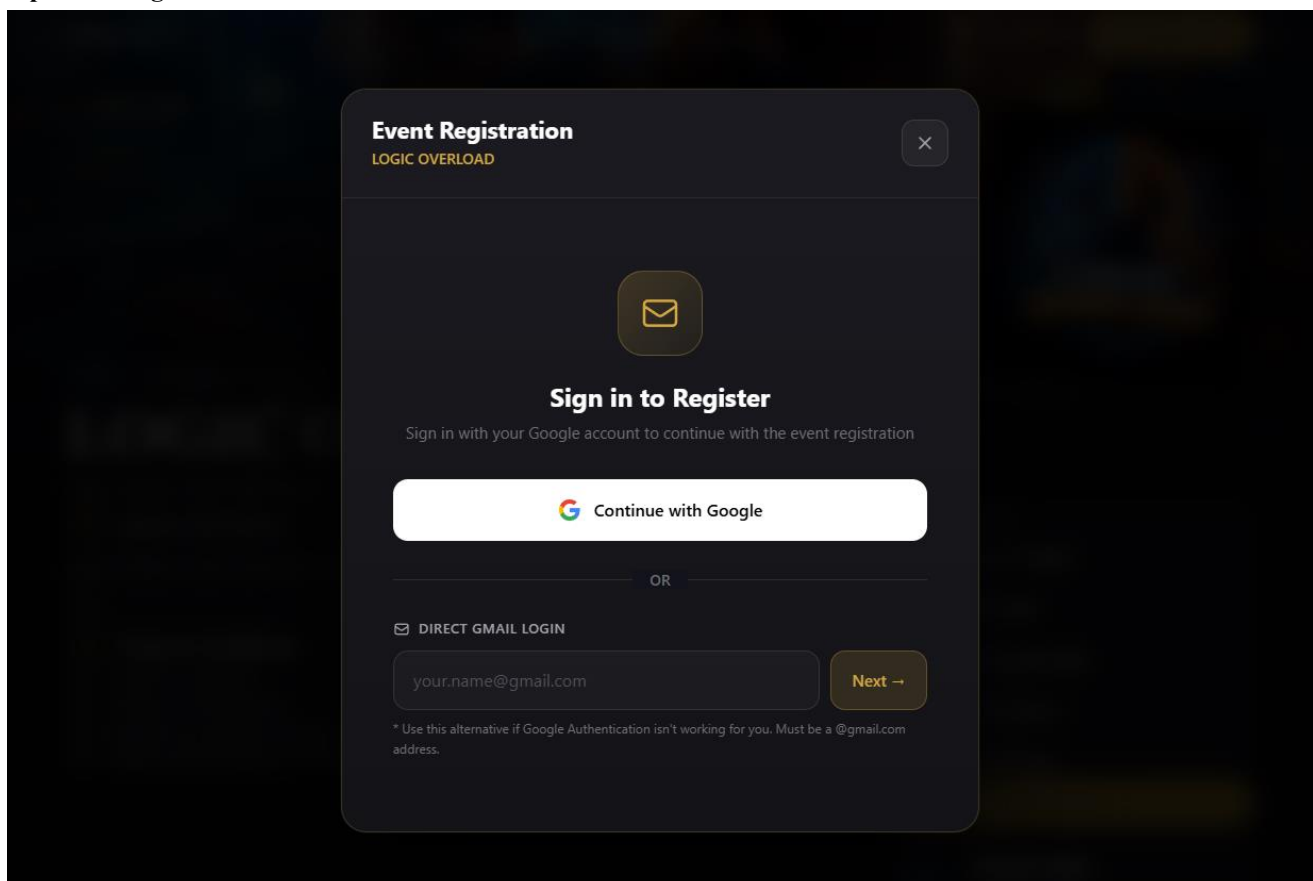


Figure 6.3: Registration Form Entry

Step 2: Adding Team Members



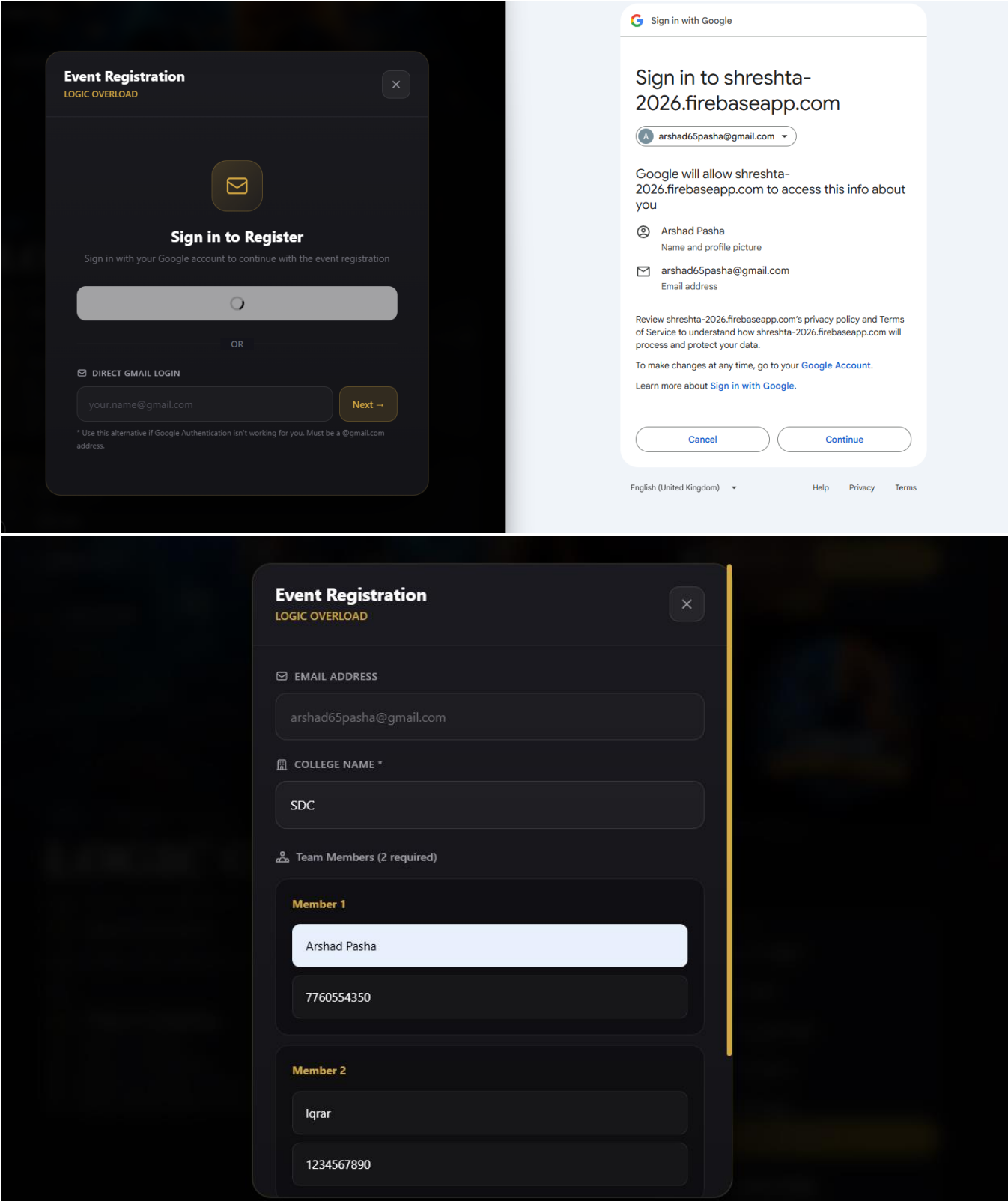
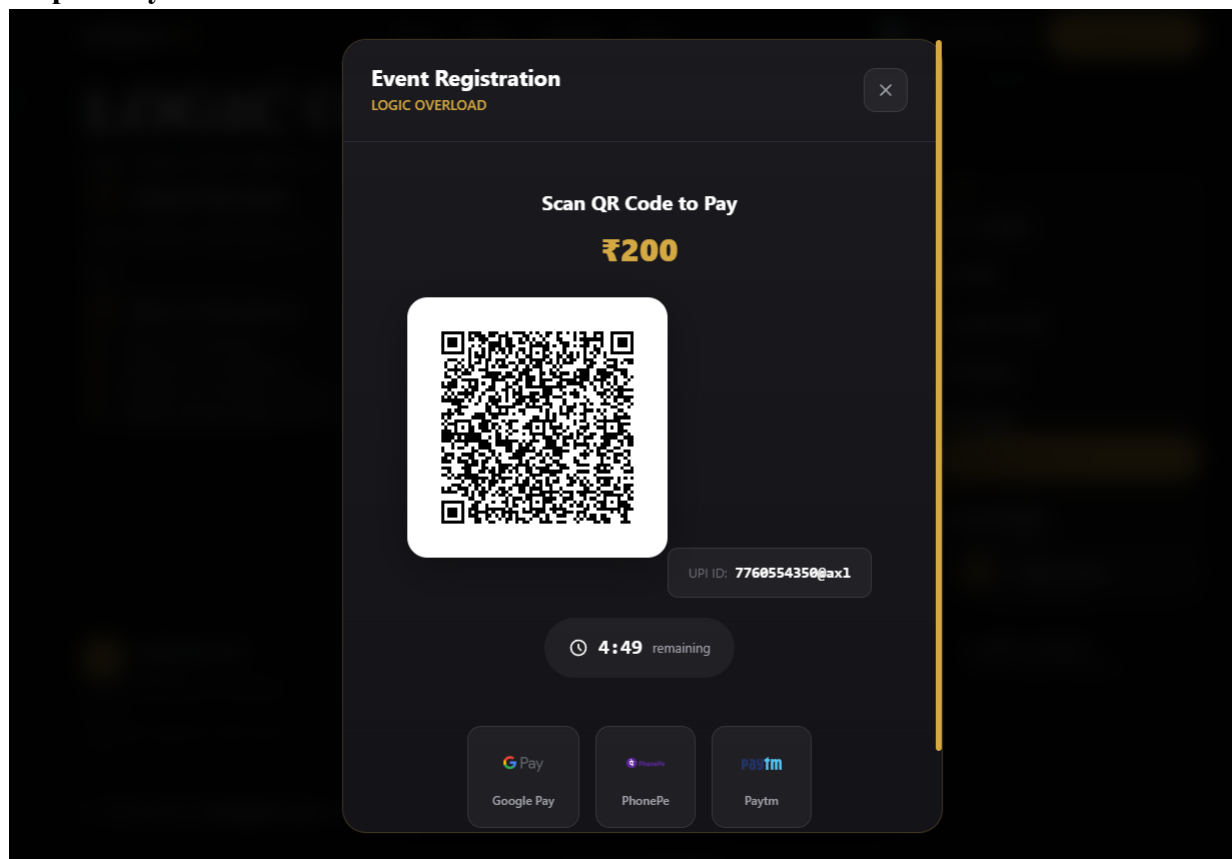


Figure 6.4: Adding Participants

Step 3: Payments and UTR Verification



Step 4: Submission and Confirmation

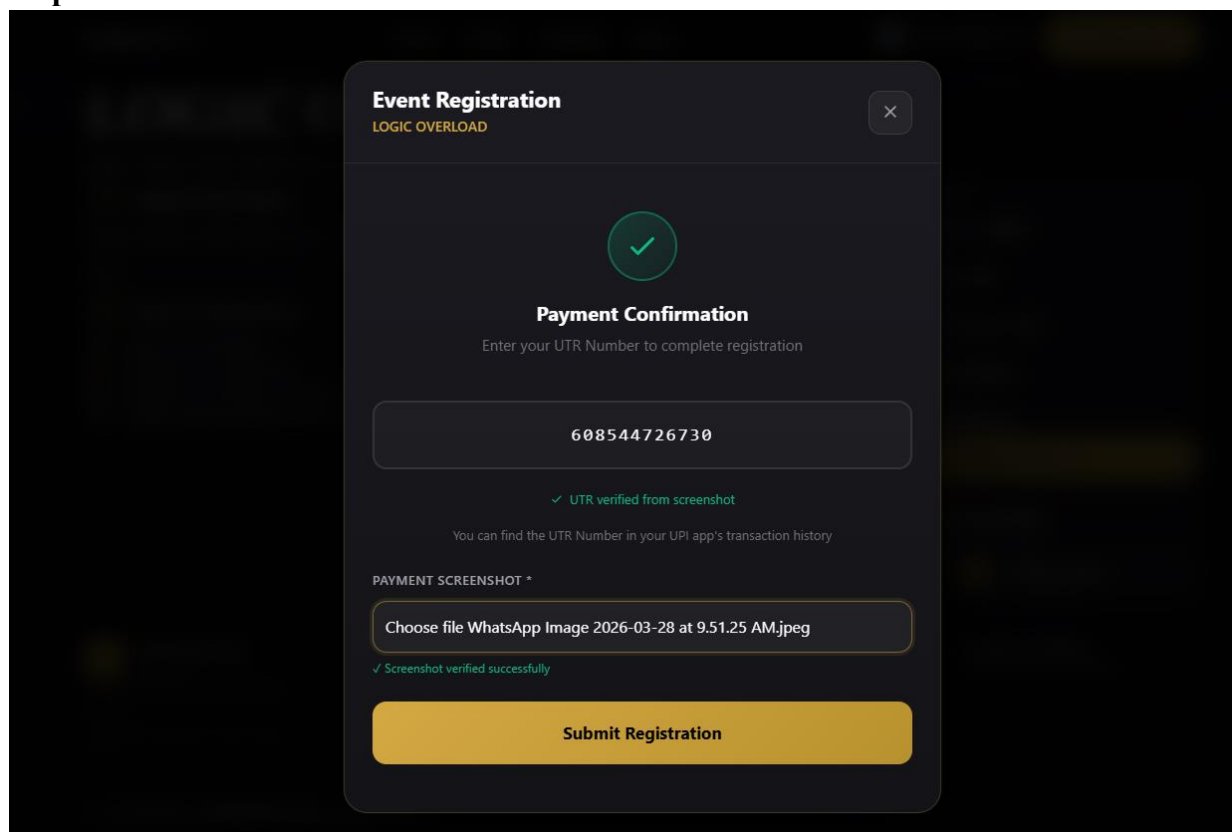


Figure 6.5: Payment and UTR Entry

6.4.2 Admin & Coordinator Dashboard Workflow

Admin Login Panel

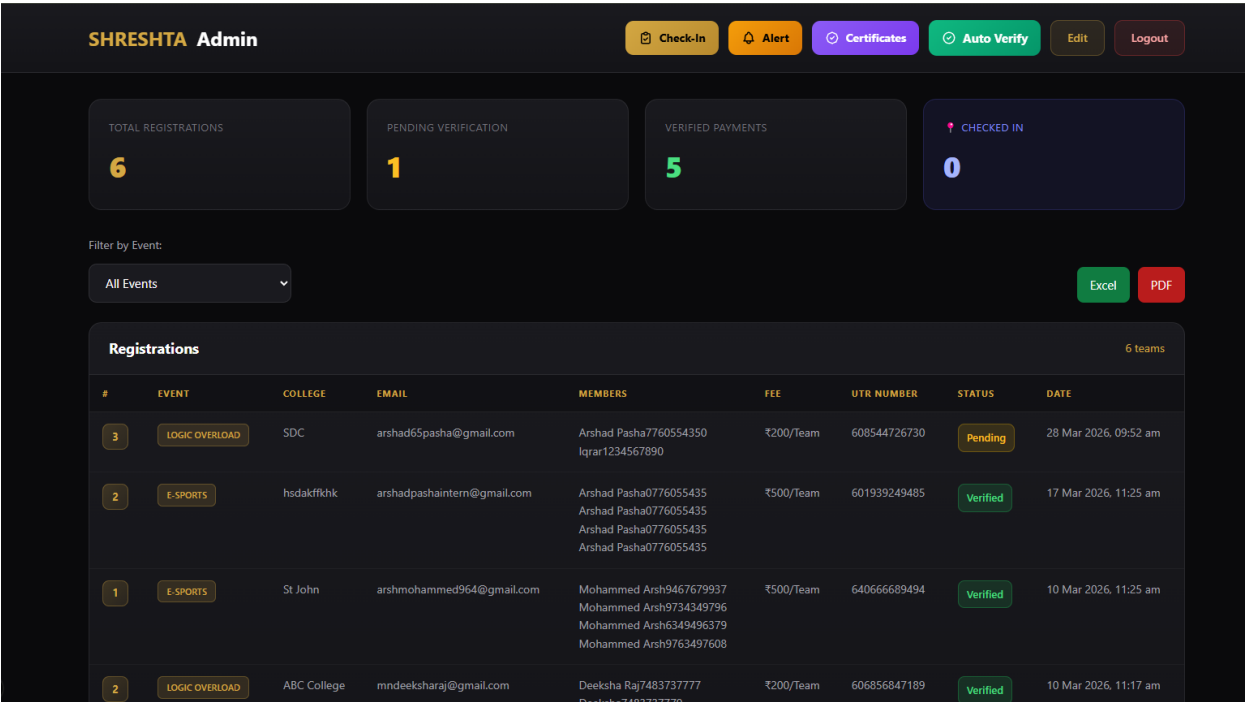


Figure 6.7: Secure Admin Login

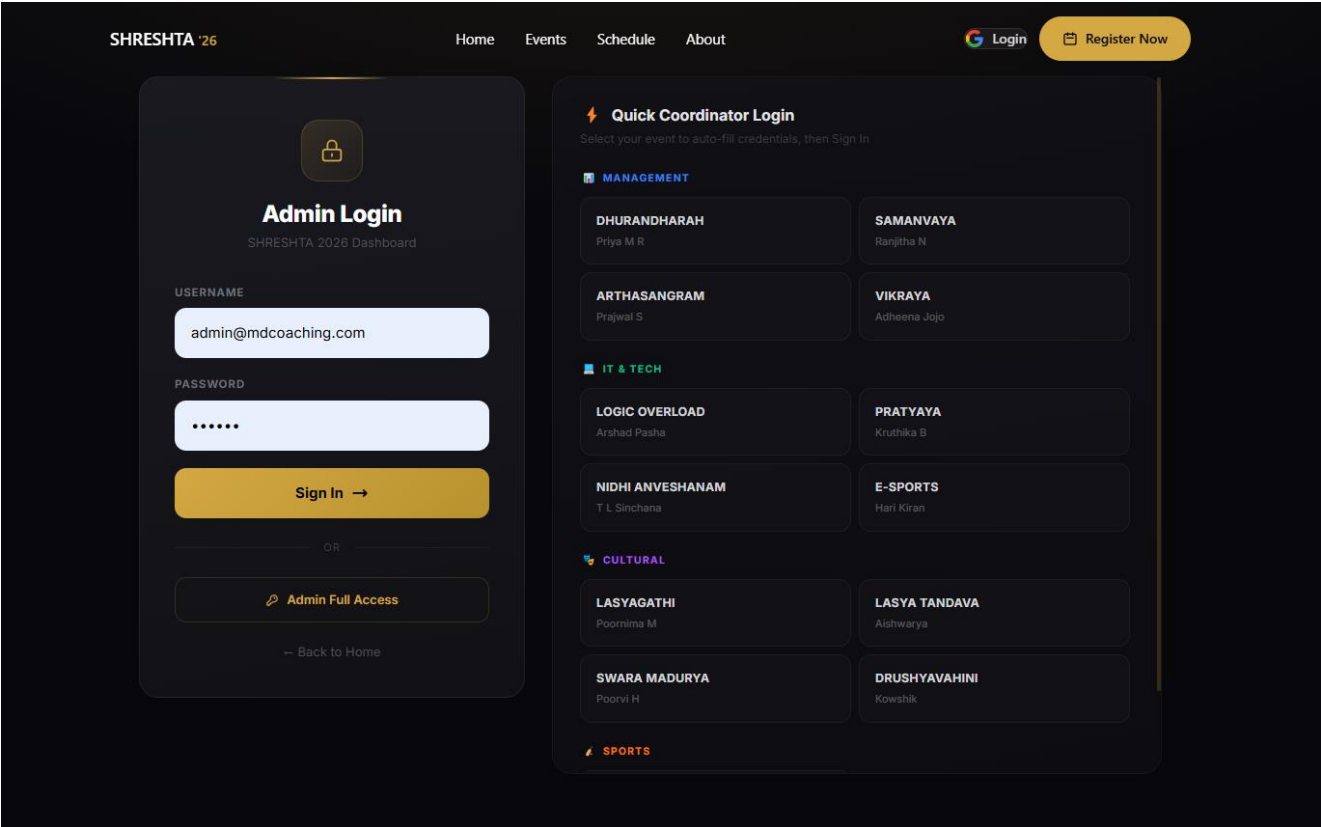


Figure 6.8: Centralized Dashboard Hub

Coordinator Dashboard View

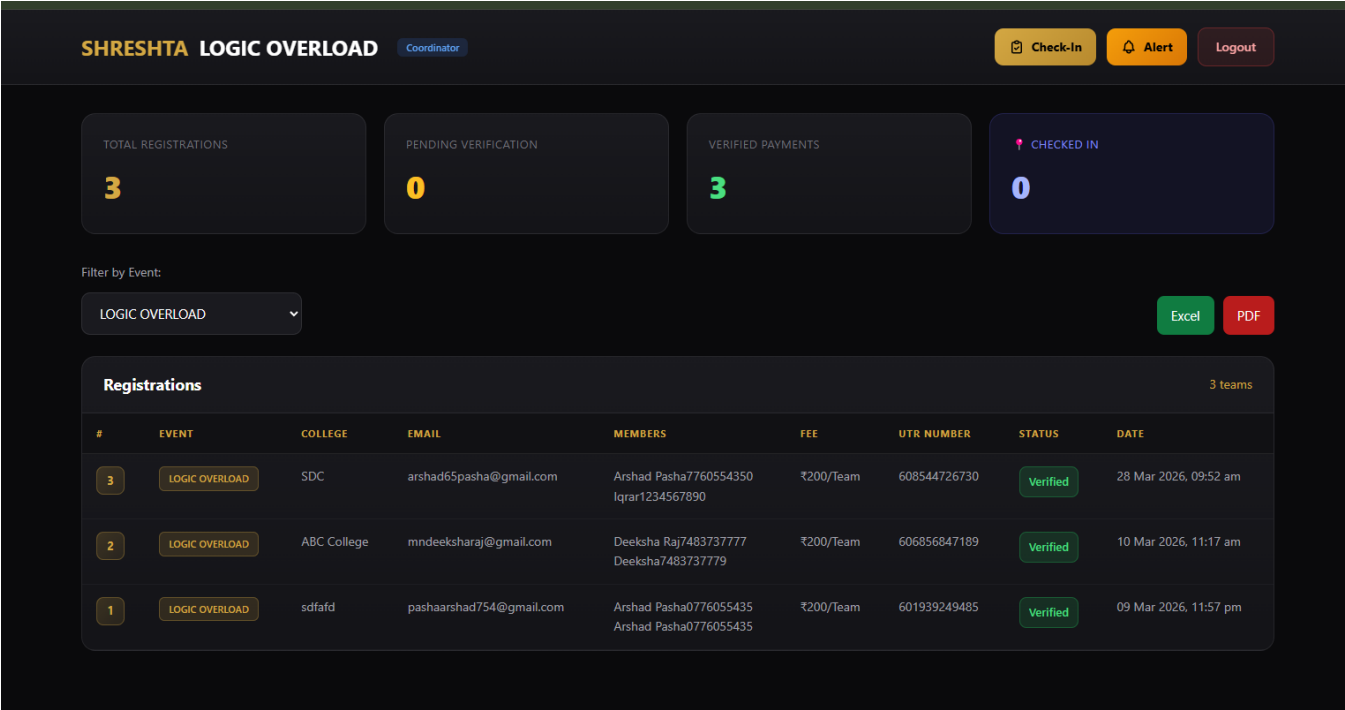
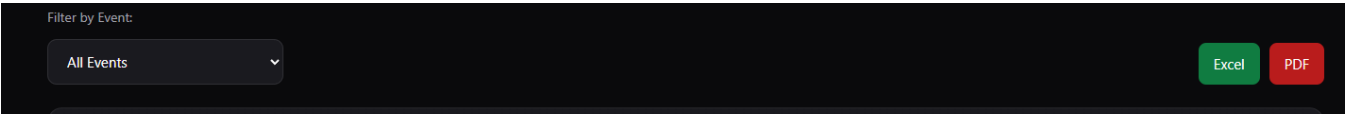


Figure 6.9: Coordinator Specific Dashboard

6.4.3 Data Export functionality (Excel/PDF)



Export Options

Select fields to include in the EXCEL file.

☒ Basic Info (ID, Event, College, Email, Members)

☒ Registration Fee

☒ UTR Number

☒ Payment Status

☒ Registration Date

Cancel

Download EXCEL

SHRESHTA 2026 - Registrations

Generated on: 2026-03-28

LOGIC OVERLOAD

ID	College	Email	M1	M2	M3	M4	M5	Fee	UTR	Status
3	SDC	arshad65pasha@gmail.com	Anshad Pasha 7760554350	Iqar 1234567890	-	-	-	* 2 0 0 / T e a m	608544726730	completed
2	ABC College	mndeeksharaj@gmail.com	Deeksha Raj 7483737777	Deeksha 7483737779	-	-	-	* 2 0 0 / T e a m	606856847189	completed
1	sdfeld	pashaanshad754@gmail.com	Anshad Pasha 0776055435	Anshad Pasha 0776055435	-	-	-	* 2 0 0 / T e a m	601939249485	completed

E-SPORTS

ID	College	Email	M1	M2	M3	M4	M5	Fee	UTR	Status
2	hedeakfshh	arshadpashantem@gmail.com	Anshad Pasha 0776055435	Anshad Pasha 0776055435	Anshad Pasha 0776055435	Anshad Pasha 0776055435	-	* 5 0 0 / T e a m	601939249485	completed
1	St John	arshmohammed964@gmail.com	Mohammed Ansh 9467679937	Mohammed Ansh 9734349796	Mohammed Ansh 6349496379	Mohammed Ansh 9763497808	-	* 5 0 0 / T e a m	640666689494	completed

PRATYAYA

ID	College	Email	M1	M2	M3	M4	M5	Fee	UTR	Status
1	SDC	dhanushivanna23@gmail.com	Dhanushree 6649664984	Reethu 9486464884	-	-	-	* 2 0 0 / T e a m	606856647189	completed

Figure 6.10: Data Extraction to Excel & PDF

6.4.4 Real-time Check-In Desk

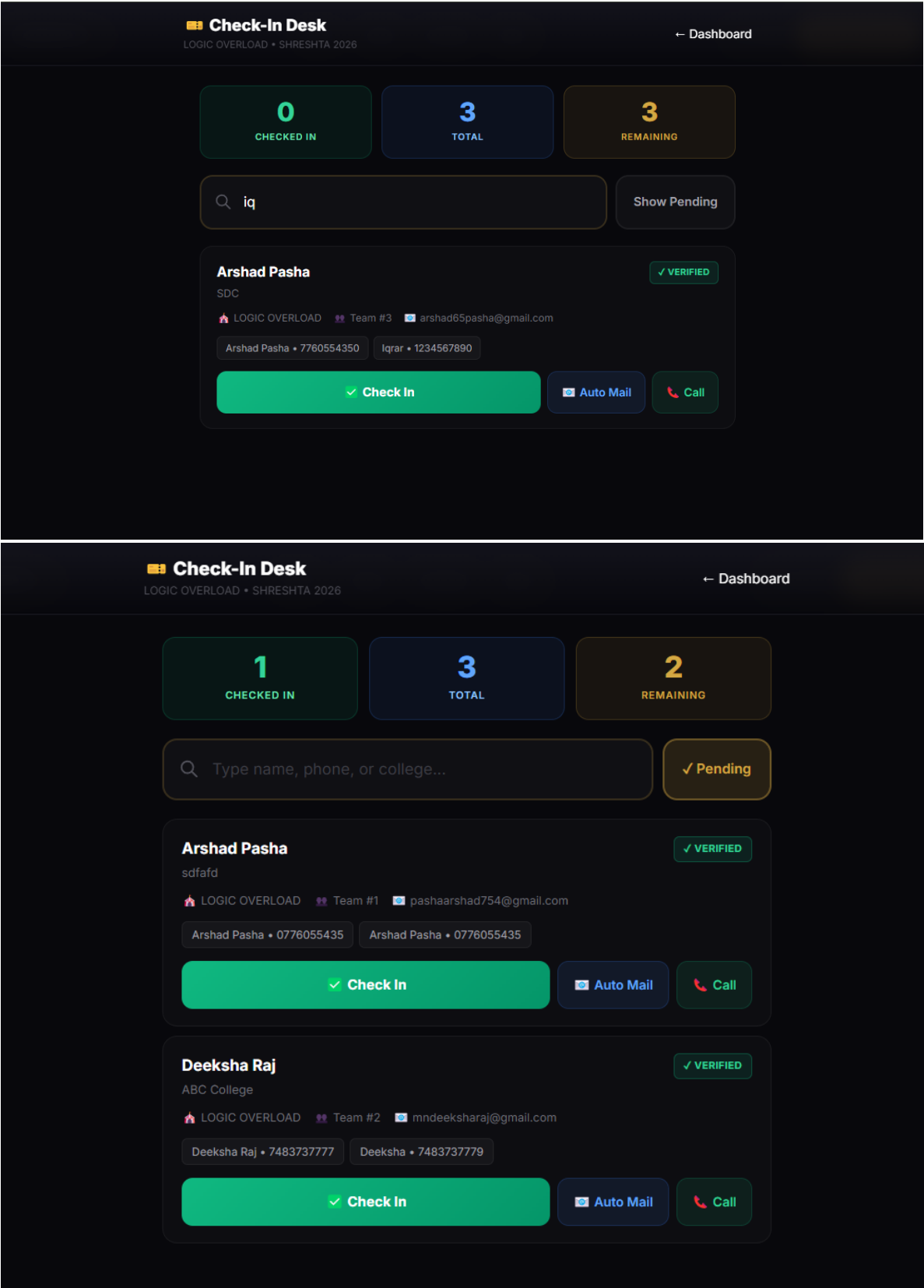


Figure 6.11: Check-in Interface and "Show Pending" Tools

6.4.5 Automated Email Verification & Certificates

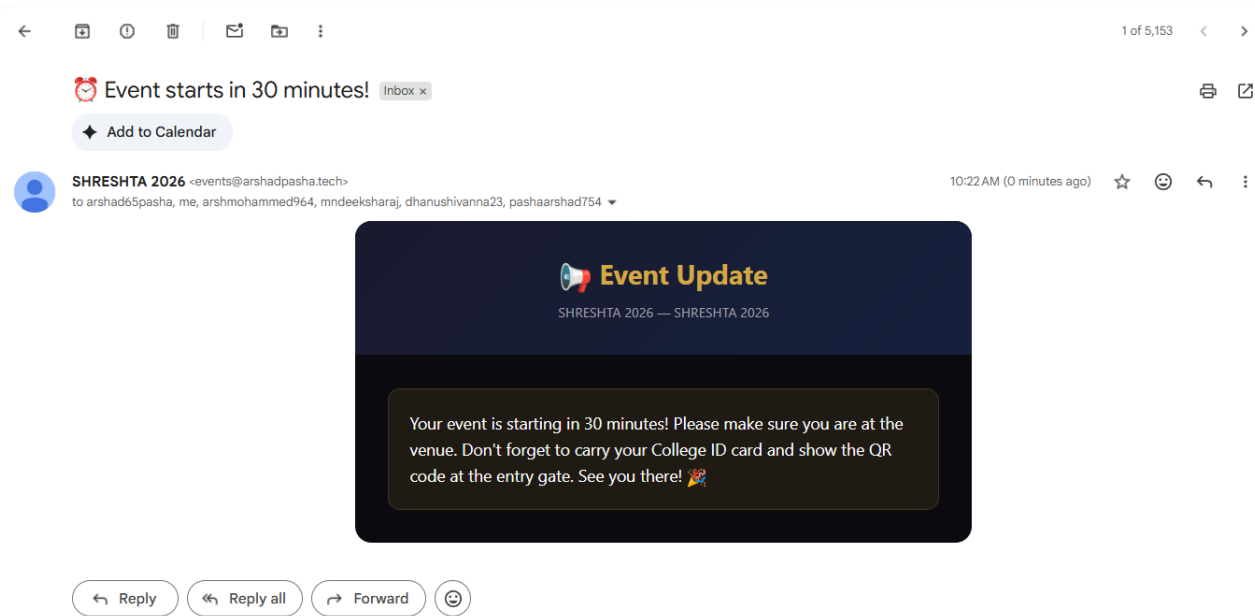
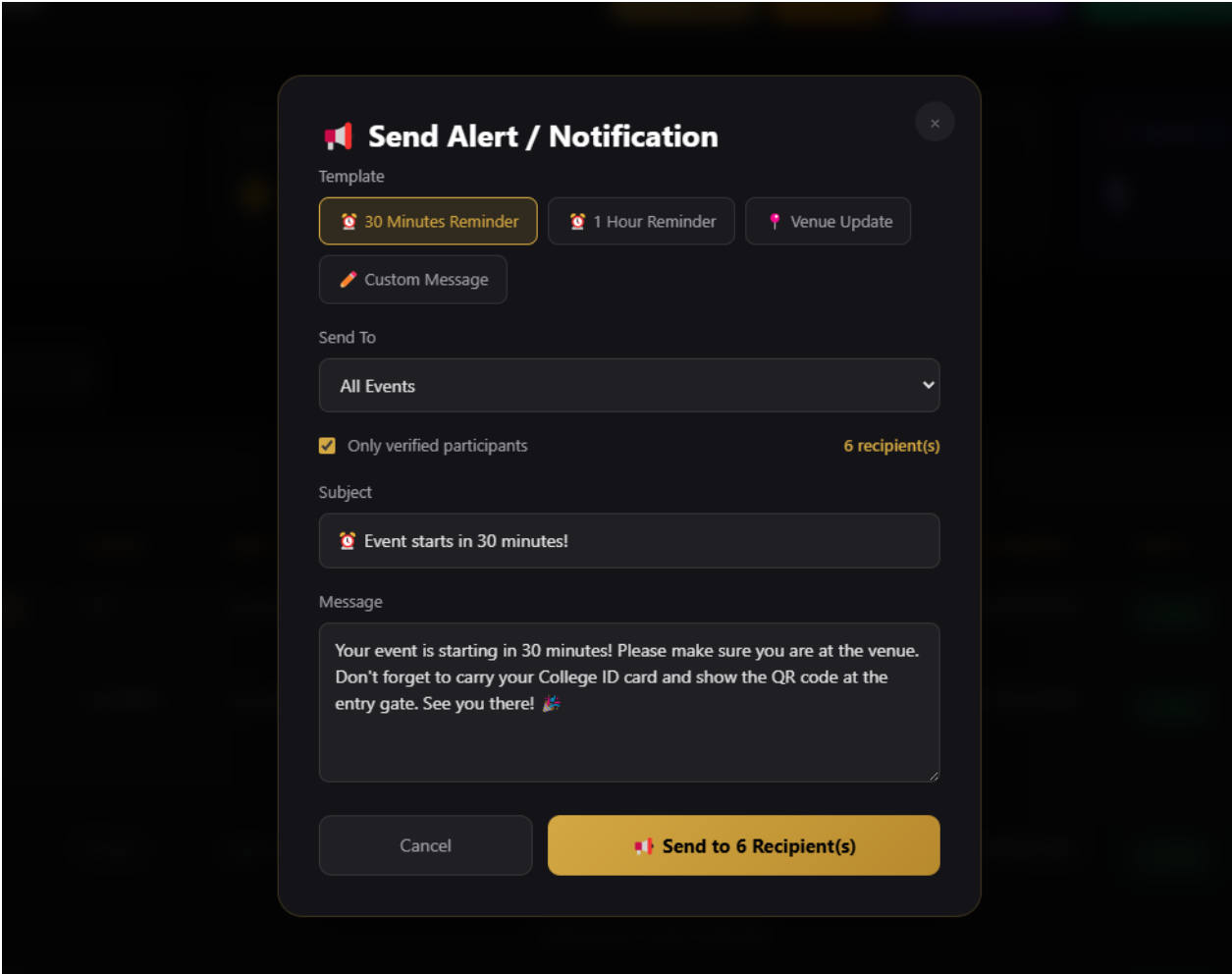
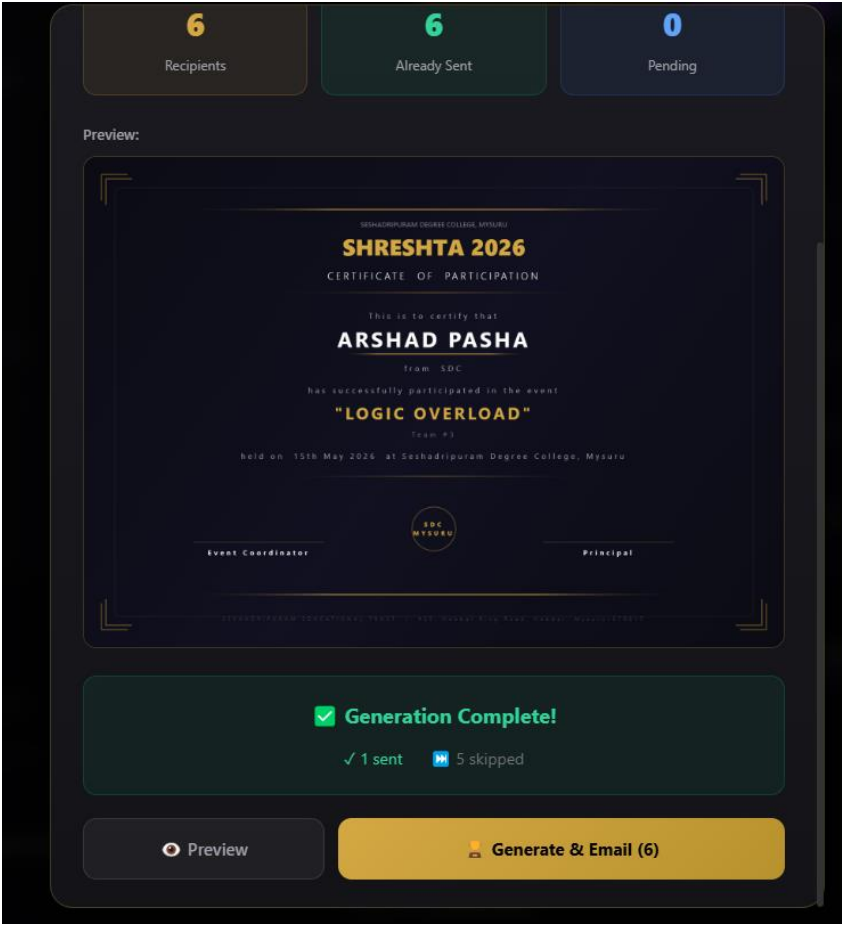
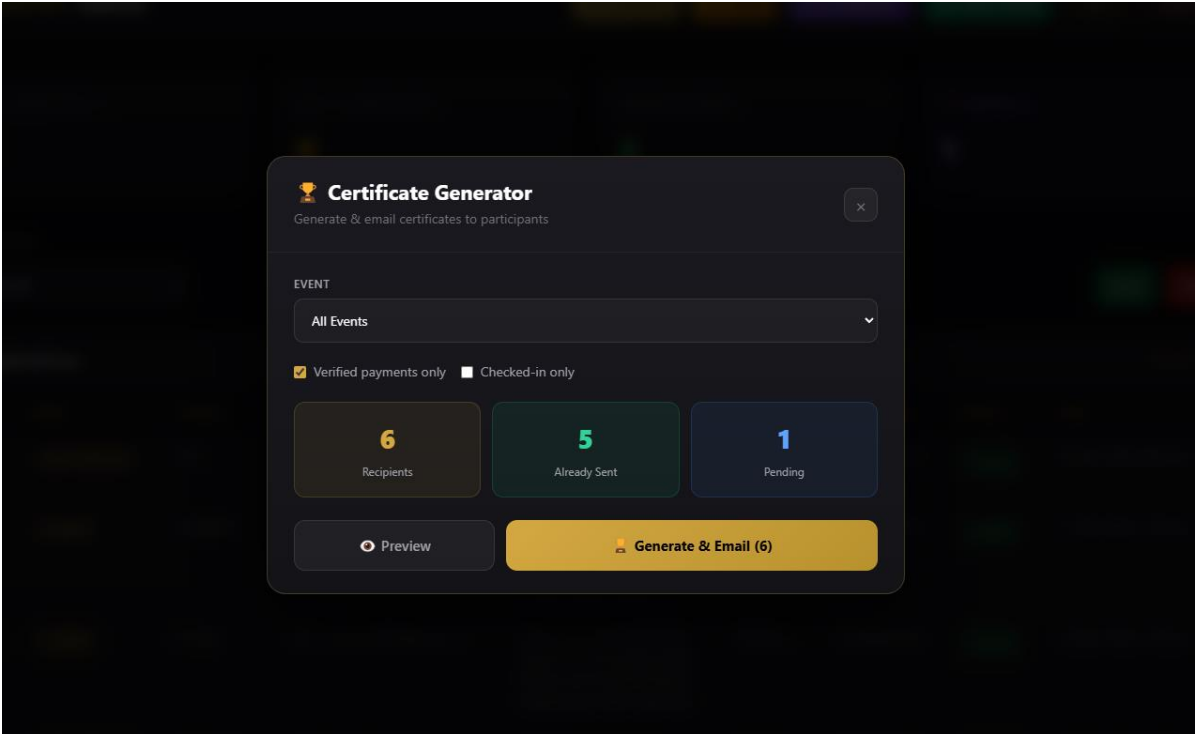


Figure 6.12: Automated Email Delivery via Resend API



13: Canvas Certificate Generation & Dispatch

CHAPTER 7

CONCLUSION & FUTURE ENHANCEMENT

7.1 Conclusion

The Smart Fest Management System has been successfully designed and developed to address the limitations of traditional fest management methods. The system provides a centralized and automated platform for managing events, registrations, and payment verification efficiently.

The application simplifies the process of event registration by allowing users to register online and submit payment details easily. It reduces manual work, minimizes errors, and improves transparency in handling participant data and transactions. The admin and event coordinators are provided with a dedicated dashboard to manage events, verify payments, and monitor overall system activities.

The system is developed using modern technologies such as React.js, Next.js, and Firebase, which ensure real-time data synchronization, scalability, and secure data handling. The modular architecture of the system makes it easy to maintain and extend in the future.

Through proper implementation and testing, the system has proven to be reliable, efficient, and user-friendly. It successfully meets both functional and non-functional requirements and performs well under different conditions.

Overall, the Smart Fest Management System provides an effective solution for organizing and managing college fests in a digital and structured manner.

Future enhancement

CHAPTER 8

BIBLIOGRAPHY

1. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.
2. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.
3. Freeman, A. (2019). *Pro React 16*. Apress.
4. Banks, A., & Porcello, E. (2020). *Learning React: Functional Web Development with React and Redux*. O'Reilly Media.
5. Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to build high-performance network programs. *IEEE Internet Computing*, 14(6), 80–83.
6. Google Firebase. (2023). Firebase Documentation. Retrieved from <https://firebase.google.com/docs>
7. Vercel Inc. (2023). Next.js Documentation. Retrieved from <https://nextjs.org/docs>
8. Mozilla Developer Network (MDN). (2023). Web Development Documentation. Retrieved from <https://developer.mozilla.org>
9. W3Schools. (2023). Web Development Tutorials. Retrieved from <https://www.w3schools.com>
10. GitHub Inc. (2023). GitHub Documentation. Retrieved from <https://docs.github.com>
11. Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures. *Doctoral Dissertation*, University of California, Irvine.
12. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.
13. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill
14. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
15. Laudon, K. C., & Laudon, J. P. (2020). *Management Information Systems* (16th ed.). Pearson.

16. Turban, E., Pollard, C., & Wood, G. (2018). *Information Technology for Management*. Wiley.
17. Bass, L., Clements, P., & Kazman, R. (2012). *Software Architecture in Practice* (3rd ed.). Addison-Wesley.
18. Preece, J., Rogers, Y., & Sharp, H. (2019). *Interaction Design: Beyond Human-Computer Interaction*. Wiley.
19. Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., Elmqvist, N., & Diakopoulos, N. (2016). *Designing the User Interface*. Pearson.
20. Krug, S. (2014). *Don't Make Me Think: A Common Sense Approach to Web Usability*. New Riders.
21. Firebase Team. (2022). Cloud Firestore Documentation. Retrieved from <https://firebase.google.com/docs/firestore>
22. ECMAScript Language Specification. (2023). JavaScript Standards. Retrieved from <https://tc39.es>
23. Oracle. (2021). Database Design Concepts. Retrieved from <https://docs.oracle.com>
24. Microsoft. (2023). Web Development Best Practices. Retrieved from <https://learn.microsoft.com>
25. ISO/IEC. (2011). *Software Engineering — System and Software Quality Requirements and Evaluation (SQuaRE)*.